

Data Structures and Algorithms

GÉZA HORVÁTH

Ninth Lecture

Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 **B-trees.**
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

Silicon memory vs. disks

- Although disks are cheaper and have higher capacity than main memory, they are much, much slower because they have moving mechanical parts.
- In order to amortize the time spent waiting for mechanical movements, disks access not just one item but several at a time. Information is divided into a number of equal-sized pages of bits, and each disk read or write is of one or more entire pages.
- Often, accessing a page of information and reading it from a disk takes longer than examining all the information read. For this reason, we shall look separately at the two principal components of the running time:
 - the number of disk accesses, and
 - the CPU (computing) time.

B-tree

- B-trees are search trees designed to work well on disks or other direct-access secondary storage devices.
- B-trees are not binary search trees. They are better at minimizing disk I/O operations.
- Because disks operate much more slowly than random-access memory, we measure the performance of B-trees not only by how much computing time the operations consume but also by how many disk accesses they perform. For each B-tree operation, the number of disk accesses increases with the height of the B-tree, but B-tree operations keep the height low.
- Many database systems use B-trees, or variants of B-trees, to store information.

B-tree – example

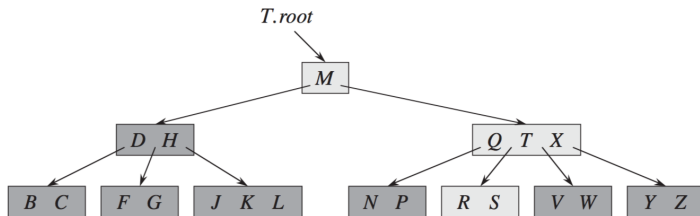
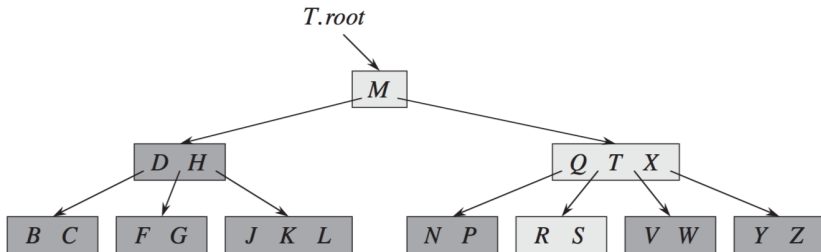


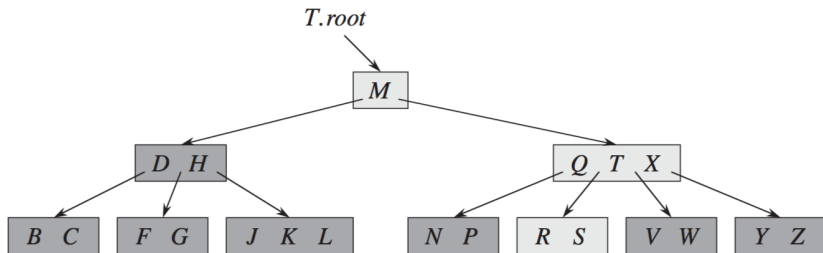
Figure 18.1 A B-tree whose keys are the consonants of English. An internal node x containing $x.n$ keys has $x.n + 1$ children. All leaves are at the same depth in the tree. The lightly shaded nodes are examined in a search for the letter R .

B-tree



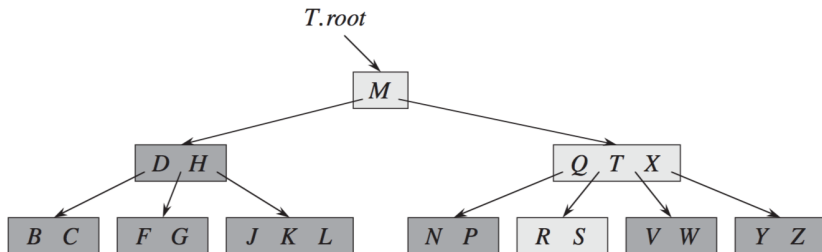
In a typical B-tree application, the amount of data handled is so large that all the data do not fit into main memory at once. The B-tree algorithms copy selected pages from disk into main memory as needed and write back onto disk the pages that have changed. B-tree algorithms keep only a constant number of pages in main memory at any time; thus, the size of main memory does not limit the size of B-trees that can be handled.

B-tree



Since in most systems the running time of a B-tree algorithm depends primarily on the number of DISK-READ and DISK-WRITE operations it performs, we typically want each of these operations to read or write as much information as possible. Thus, a B-tree node is usually as large as a whole disk page, and this size limits the number of children a B-tree node can have.

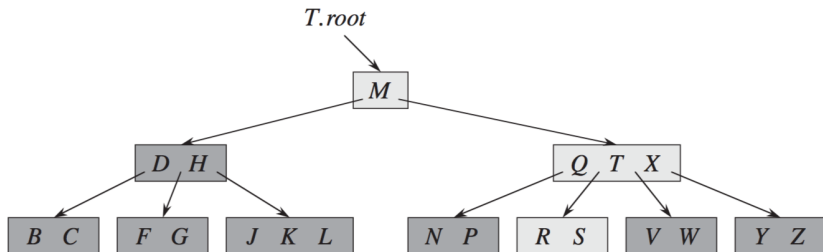
Branching factor



Definition

In tree data structures, the branching factor is the number of children at each node. (If this value is not uniform, an average branching factor can be calculated.)

Branching factor



For a large B-tree stored on a disk, we often see branching factors between 50 and 2000, depending on the size of a key relative to the size of a page. A large branching factor dramatically reduces both the height of the tree and the number of disk accesses required to find any key.

Branching factor

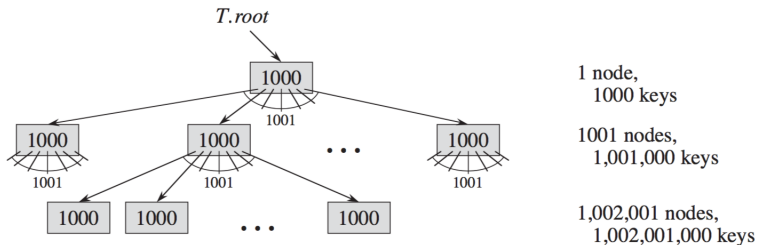
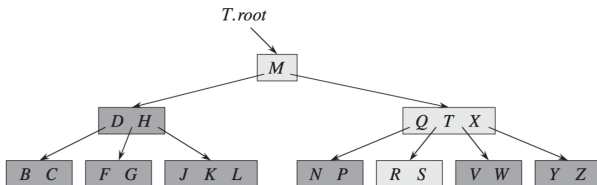


Figure 18.3 A B-tree of height 2 containing over one billion keys. Shown inside each node x is $x.n$, the number of keys in x . Each internal node and leaf contains 1000 keys. This B-tree has 1001 nodes at depth 1 and over one million leaves at depth 2.

Figure 18.3 shows a B-tree with a branching factor of 1001 and height 2 that can store over one billion keys; nevertheless, since we can keep the root node permanently in main memory, we can find any key in this tree by making at most only two disk accesses.

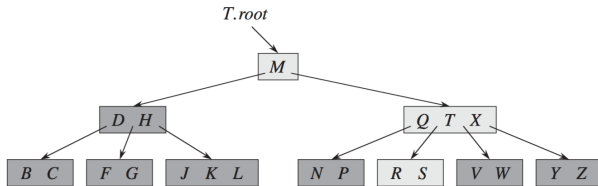
B-tree – definition



A **B-tree** T is a rooted tree (whose root is $T.root$) having the following properties:

- Every node x has the following attributes:
 - $x.n$, the number of keys currently stored in node x ,
 - the $x.n$ keys themselves, $x.key_1, x.key_2, \dots, x.key_{x.n}$, stored in nondecreasing order, so that $x.key_1 \leq x.key_2 \leq \dots \leq x.key_{x.n}$,
 - $x.leaf$, a boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.
- Each internal node x also contains $x.n + 1$ pointers $x.c_1, x.c_2, \dots, x.c_{x.n+1}$ to its children. Leaf nodes have no children, and so their c_i attributes are undefined.

B-tree – definition

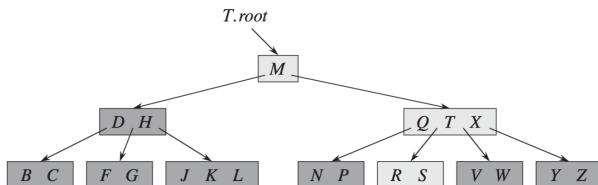


3. The keys $x.key_i$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root $x.c_i$, then

$$k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \dots \leq x.key_{x.n} \leq k_{x.n+1} .$$

4. All leaves have the same depth, which is the tree's height h .

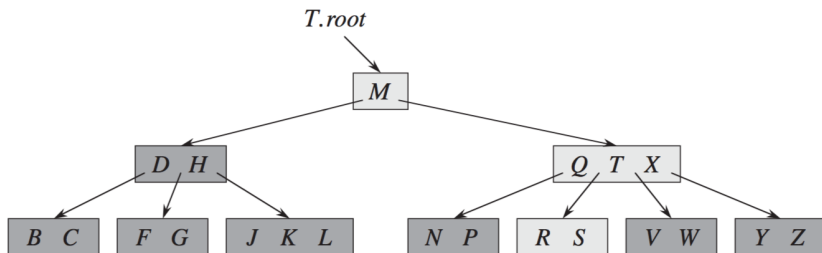
B-tree – definition



5. Nodes have lower and upper bounds on the number of keys they can contain. We express these bounds in terms of a fixed integer $t \geq 2$ called the **minimum degree** of the B-tree:
 - a. Every node other than the root must have at least $t - 1$ keys. Every internal node other than the root thus has at least t children. If the tree is nonempty, the root must have at least one key.
 - b. Every node may contain at most $2t - 1$ keys. Therefore, an internal node may have at most $2t$ children. We say that a node is **full** if it contains exactly $2t - 1$ keys.

The simplest B-tree occurs when $t = 2$. Every internal node then has either 2, 3, or 4 children, and we have a **2-3-4 tree**. In practice, however, much larger values of t yield B-trees with smaller height.

B-tree – height



Theorem

If $n \geq 1$, then for any n -key B-tree T of height h and minimum degree $t \geq 2$,

$$h \leq \log_t \frac{n+1}{2}.$$

B-tree – data structure operations

Implemented data structure operations:

- **create** data structure – B-TREE-CREATE
- **modify** data structure
 - add elements – B-TREE-INSERT
 - delete elements – B-TREE-DELETE
- **access** – B-TREE-SEARCH

B-TREE-CREATE(T)

```
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.\text{leaf} = \text{TRUE}$ 
3   $x.n = 0$ 
4   $\text{DISK-WRITE}(x)$ 
5   $T.\text{root} = x$ 
```

B-TREE-CREATE requires $O(1)$ disk operations and $O(1)$ CPU time.

B-tree – search

B-TREE-SEARCH(x, k)

```

1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return ( $x, i$ )
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )

```

The B-TREE-SEARCH procedure accesses $\mathcal{O}(h) = \mathcal{O}(\log_t n)$ disk pages, where h is the height of the B-tree and n is the number of keys in the B-tree. Since $x.n < 2 * t$, the while loop of lines 2-3 takes $\mathcal{O}(t)$ time within each node, and the total CPU time is $\mathcal{O}(t * h) = \mathcal{O}(t * \log_t n)$.

B-tree – insertion

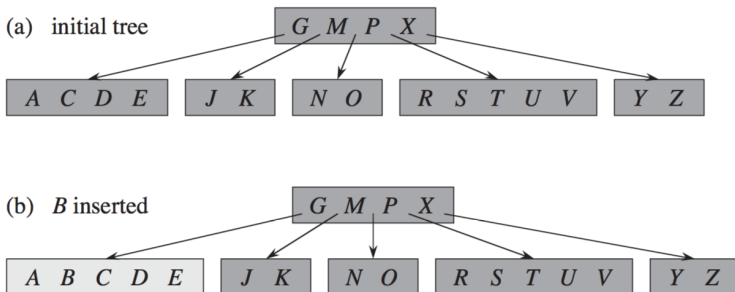


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3, so a node can hold at most 5 keys. Nodes that are modified by the insertion process are lightly shaded. **(a)** The initial tree for this example. **(b)** The result of inserting B into the initial tree; this is a simple insertion into a leaf node. **(c)** The result of inserting Q into the previous tree. The node $RSTUV$ splits into two nodes containing RS and UV , the key T moves up to the root, and Q is inserted in the leftmost of the two halves (the RS node). **(d)** The result of inserting L into the previous tree. The root splits right away, since it is full, and the B-tree grows in height by one. Then L is inserted into the leaf containing JK . **(e)** The result of inserting F into the previous tree. The node $ABCDE$ splits before F is inserted into the rightmost of the two halves (the DE node).

B-tree – insertion

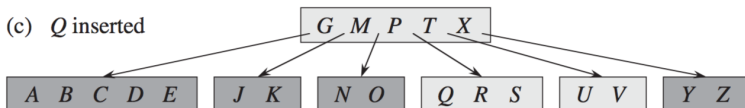
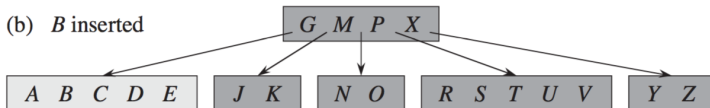
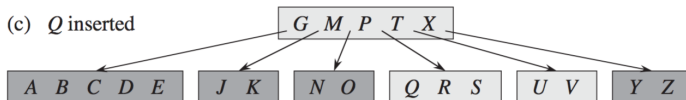


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3, so a node can hold at most 5 keys. Nodes that are modified by the insertion process are lightly shaded. (a) The initial tree for this example. (b) The result of inserting *B* into the initial tree; this is a simple insertion into a leaf node. (c) The result of inserting *Q* into the previous tree. The node *RSTUV* splits into two nodes containing *RS* and *UV*, the key *T* moves up to the root, and *Q* is inserted in the leftmost of the two halves (the *RS* node). (d) The result of inserting *L* into the previous tree. The root splits right away, since it is full, and the B-tree grows in height by one. Then *L* is inserted into the leaf containing *JK*. (e) The result of inserting *F* into the previous tree. The node *ABCDE* splits before *F* is inserted into the rightmost of the two halves (the *DE* node).

B-tree – insertion

(c) *Q* inserted



(d) *L* inserted

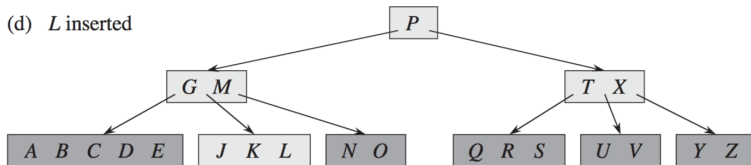


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3, so a node can hold at most 5 keys. Nodes that are modified by the insertion process are lightly shaded. (a) The initial tree for this example. (b) The result of inserting *B* into the initial tree; this is a simple insertion into a leaf node. (c) The result of inserting *Q* into the previous tree. The node *RSTUV* splits into two nodes containing *RS* and *UV*, the key *T* moves up to the root, and *Q* is inserted in the leftmost of the two halves (the *RS* node). (d) The result of inserting *L* into the previous tree. The root splits right away, since it is full, and the B-tree grows in height by one. Then *L* is inserted into the leaf containing *JK*. (e) The result of inserting *F* into the previous tree. The node *ABCDE* splits before *F* is inserted into the rightmost of the two halves (the *DE* node).

B-tree – insertion

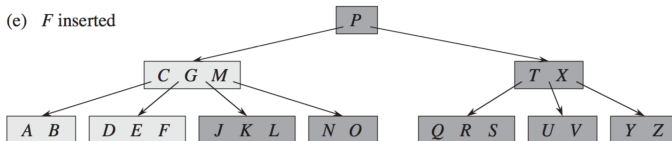
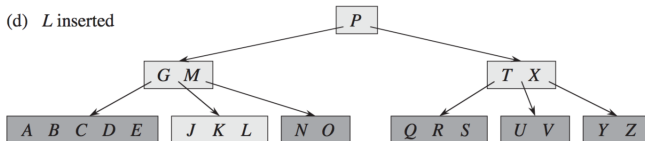


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3, so a node can hold at most 5 keys. Nodes that are modified by the insertion process are lightly shaded. (a) The initial tree for this example. (b) The result of inserting B into the initial tree; this is a simple insertion into a leaf node. (c) The result of inserting Q into the previous tree. The node $RSTUV$ splits into two nodes containing RS and UV , the key T moves up to the root, and Q is inserted in the leftmost of the two halves (the RS node). (d) The result of inserting L into the previous tree. The root splits right away, since it is full, and the B-tree grows in height by one. Then L is inserted into the leaf containing JK . (e) The result of inserting F into the previous tree. The node $ABCDE$ splits before F is inserted into the rightmost of the two halves (the DE node).

B-tree – insertion

B-TREE-INSERT(T, k)

```

1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{ALLOCATE-NODE}()$ 
4       $T.root = s$ 
5       $s.leaf = \text{FALSE}$ 
6       $s.n = 0$ 
7       $s.c_1 = r$ 
8      B-TREE-SPLIT-CHILD( $s, 1$ )
9      B-TREE-INSERT-NONFULL( $s, k$ )
10 else B-TREE-INSERT-NONFULL( $r, k$ )
```

For a B-tree of height h , **B-TREE-INSERT** performs $\mathcal{O}(h)$ disk accesses, since only $\mathcal{O}(1)$ **DISK-READ** and **DISK-WRITE** operations occur between calls to **B-TREE-INSERT-NONFULL**. The total CPU time used is $\mathcal{O}(t * h) = \mathcal{O}(t * \log_t n)$.

B-tree – insertion

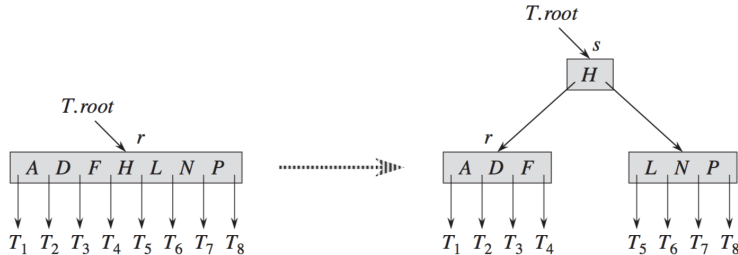


Figure 18.6 Splitting the root with $t = 4$. Root node r splits in two, and a new root node s is created. The new root contains the median key of r and has the two halves of r as children. The B-tree grows in height by one when the root is split.

B-tree – insertion

B-TREE-SPLIT-CHILD(x, i)

```
1   $z = \text{ALLOCATE-NODE}()$ 
2   $y = x.c_i$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$ 
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$ 
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$ 
11 for  $j = x.n + 1$  downto  $i + 1$ 
12      $x.c_{j+1} = x.c_j$ 
13  $x.c_{i+1} = z$ 
14 for  $j = x.n$  downto  $i$ 
15      $x.\text{key}_{j+1} = x.\text{key}_j$ 
16  $x.\text{key}_i = y.\text{key}_t$ 
17  $x.n = x.n + 1$ 
18 DISK-WRITE( $y$ )
19 DISK-WRITE( $z$ )
20 DISK-WRITE( $x$ )
```

B-tree – insertion

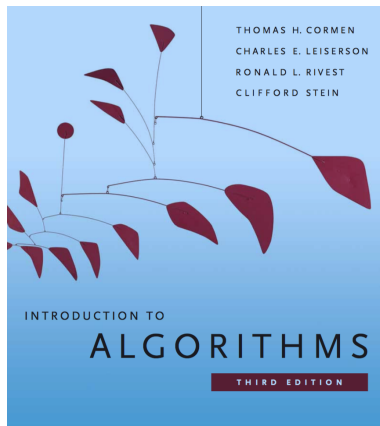
B-TREE-INSERT-NONFULL(x, k)

```

1   $i = x.n$ 
2  if  $x.leaf$ 
3      while  $i \geq 1$  and  $k < x.key_i$ 
4           $x.key_{i+1} = x.key_i$ 
5           $i = i - 1$ 
6       $x.key_{i+1} = k$ 
7       $x.n = x.n + 1$ 
8      DISK-WRITE( $x$ )
9  else while  $i \geq 1$  and  $k < x.key_i$ 
10       $i = i - 1$ 
11       $i = i + 1$ 
12      DISK-READ( $x.c_i$ )
13      if  $x.c_i.n == 2t - 1$ 
14          B-TREE-SPLIT-CHILD( $x, i$ )
15          if  $k > x.key_i$ 
16               $i = i + 1$ 
17      B-TREE-INSERT-NONFULL( $x.c_i, k$ )

```


References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.